

COMP 3033 – Design Patterns with C++(SP2 2023)

Course	Design Patterns with C++
Name	Gitae Bae
Email	baegy002@mymail.unisa.edu.au
Identifier	110310861
Date	Sunday, 16/04/2023

Programming Assignment 1- Sushi Slam!

[Gitae Bae]

[16/04/2023]

Contents

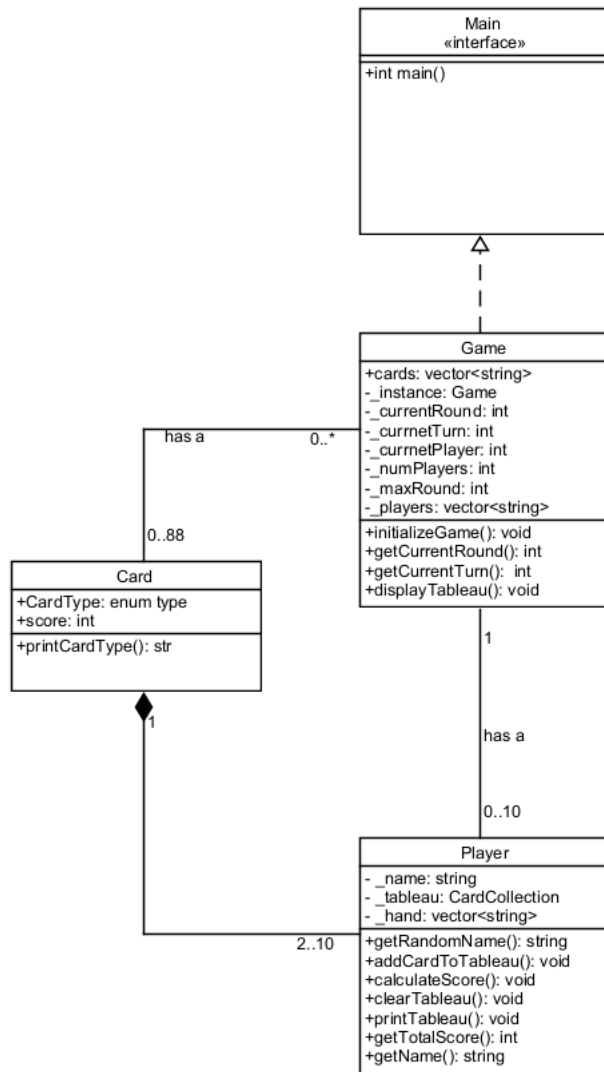
1. Class Diagram3

2. Design Description4

 2.1 Attribute and Method4

 2.2 Relationship5

1. Class Diagram



2. Design Description

2.1 Attribute and Method

Game, Card, and Player classes are basic and are a kind of framework in object-oriented programming that defines variables and methods to create a particular object. It consists of methods and variables to define objects. Templates allow you to decide when to use an object rather than pre-determining the type of its members when classifying an object.

Game class:

Attribute -

- Cards: This is a vector<Card*> from the Card class.
- _
_instance: This is a private attribute and static form Game Class.
- _
_gameDeck: This is a private attribute and CardCollection from the Card class.
- _
_currentRound: This is a private attribute and integer.
- _
_currentTurn: This is a private attribute and integer.
- _
_currentPlayer: This is a private attribute and integer.
- _
_numPlayers: This is a private attribute and integer.
- _
_maxRound: This is a private attribute and integer.
- _
_players: This is a vector<Player> from the Player class.

Method -

- ++initializeGame(): This method is responsible for initialising and starting a new game and void function.
- ++getCurrentRound(): This method knows the current round of the game and the integer.
- ++getCurrentTurn(): This method knows the turn in the round and the integer.
- ++displayTableau(): This method knows the current player and the integer.

Player class:

Attribute -

- _
_name: This is a private attribute and string.
- _
_tableau: This is a private attribute and CardCollection from the Card class.
- _
_hand: This is a private attribute and vector<string>.

Method-

-+getRandomName(): This method selects it from a random list and string

-+addCardToName(): When implementing player, we suggest having a function that adds a card to the player's tableau and removes it from the hand and void function.

-+addCardToTableau(): This method is responsible for adding cards to the player's current tableau and void function.

-+calculateScore(): This method is responsible for calculating a player's score for the current round and void function.

-+clearTableau(): This is responsible for clearing the player's tableau and void function.

-+printTableau(): This is responsible for printing the player's tableau to the screen and void function.

-+getTotalScore(): This method knows the player's current total score and integer.

-+getName(): This method knows the player's name, randomly initialized from a fixed list and string.

Card class:

Attribute -

-+CardType: This is a public attribute and enum type. When implementing a card, define an enum for the type of card it represents.

-+score: This is a public attribute and string.

Method-

-+printCardType(): This method is responsible for presenting the card type (i.e. printing the card type to screen) back to the player.

Main class:

Method-

-+int main(): Main interface, The main() method can appear in **any class that is part of an application**, but if the application is a complex containing multiple files, it is common to create a separate class just for main(). The main class can have any name, although typically it will just be called "Main".

2.2 Relationship

-Game & Main: Main gives help to Game.

-Game & Card: The game has a lot of cards but it has only one Game.

Gitae Bae baegy002(110310861)

-Game & Player: The game has a lot of players but it has only one Game.

-Player & Card: The card has a lot of players but it has only one Card.